GUIDE Comparison for Mathew Boyd's 2025 Pitch Selection

Phillip DeGroot

April 30, 2026

Abstract

In the competitive world of MLB baseball, data analysts devote their time to finding an edge that they can use to help their team succeed. In this paper I will be using GUIDE to create a "pitch report" for Mathew Boyd that could serve to help batters make informed predictions for what type of pitch he may throw at the plate using his 2025 pitch data which is publicly available on Baseball Savant. Because of this, I have focused on interoperability and producing classification trees that are simple enough to provide value. The purpose of this paper is to compare and contrast different statistical methods with GUIDE and find which models are easiest to interpret while lowering the misclassification rate.

First the data had to be cleaned because the raw Statcast `on_1b`/`on_2b`/`on_3b` columns contain MLBAM player IDs rather than indicators, so I collapsed them into a single three-level `base_state` variable (empty / runner not in scoring position / RISP) and excluded the raw columns in the `.dsc` file. I also had to remove columns that gave GUIDE problems, specifically the `desc` varaiable which contained detailed descriptions of what the result was from a pitch and often broke GUIDE's 200 character rule. Because my focus was on predictors of `pitch_type` and `desc` was a description of outcomes, it was easy to omit the column in total. My exploratory tree in GUIDE contained all pre-pitch predictor variables and some had driven suspicious early splits, specifically `bat_win_exp`, `if_fielding_alignment`, `of_fielding_alignment`, `batter_days_since_prev_game`, and `bat_score`. Because it is reasonable to assume that a pitcher on the mound does not consciously think about all there variables before every pitch, if at all. The only predictors I left I believed that it is reasonable to think that it may effect a pitchers decision making when choosing the next pitch which resulted in a more interpretable GUIDE tree.

To emphasize interoperability and ease of use for any hypothetical batter facing Boyd, I constrained GUIDE with the parameters of a maximum of 5 splits and a minimum of 30 observations in each terminal node. The pruned tree splits first on batter handedness `stand`, indicating the strongest single driver of Boyd's pitch mix. Against left-handed batters, the tree splits next on whether the at-bat is the batter's first of the game and for first at-bats Boyd is fastball dominant (∼52% FF) with the slider as a clear secondary (∼32%); on later at-bats he flattens his distribution dramatically, with the slider becoming the "go to" pitch and the curveball and sinker each climbing above 10%. Against right-handed batters, the tree splits on `pitch_number`: the very first pitch of an at-bat is heavily fastball driven (∼79% FF first time through the order), while subsequent pitches show much greater reliance on the changeup, particularly later in games and in deeper counts. These patterns are consistent with established baseball norms of pitchers tending to rely on the fastball early while shifting into off-speed pitch in later at-bats.

To compare the results of the GUIDE single tree, I modeled the same data on GUIDE LDA, `rpart`, `ctree`, multinomial logistic regression, and `ranger` with the last four being R packages. To make the data compatible to R, I had to select just the predictor viarables I used in GUIDE

and create a new data frame with just those while in GUIDE I was able to keep the full dataset and just "turn on and off" variables through the `.dsc`. Once the data was cleaned, I created a for loop that simulated each method 500 times and found the misclassification rate of each model and compare how they performed to GUIDE's simple single classification tree, which I used as my baseline. Comparing the trees from GUIDE, `rpart`, and `ctree` all agree on the first split being `stance`, signifying the value that a batter's stance has in determining pitch selection. Again all tree trees roughly agree with the important variables when making splits after `stance` with `balls`, `strikes`, `n_thruorder_pitcher`, and `n_prior_thisgame_player_at_bat`. This is a positive sign because we can confidently say that these predictors are valuable in determining what pitch Boyd will throw to the batter. To determine which tree made a better "fit" we need to look at the misclassification rate where GUIDE, on average, performed better than `rpart` and `ctree`. This is likely to how the different trees decide to make splits. While `rpart` and `ctree` tend to give higher values to more continuous variables because they get more splits than discrete variables, GUIDE uses χ^2 distribution so variables including but not limited to `n_prior_thisgame_player_at_bat` and `age_legacy_batter` that are high cardinality variables are given less "power". This is evident in the `ranger` model where high cardinality variables are all deemed to be more important than discrete variables on average, showing the main benefit of GUIDE. Multinomial logistic regression also fails here because it fails to point out what variables are "important" and does not capture the interaction effects. Pitches that aren't thrown as often such as `SI`(slider) and `none` have extremely low negative values due to a lack of data.

The most interesting comparison is the GUIDE simple model and the GUIDE LDA model. Viewing the `output.txt` files, we see that the simple model has a lower misclassification rate compared to LDA, which was initially shocking to me but upon further investigation, made itself clear. LDA assumes each class follows multivariate normal distribution with a shared covariance matrix. This assumption is immediatly refuted as predictors such as `balls` and `strikes` and non-continuous variables nor are they normal, hence the simple model performs better.

In conclusion, for predicting `pitch_type`, GUIDE simple performs the best due to its χ^2 distribution tactics, giving a lower misclassification rate than other tree methods. Because of this, GUIDE is much better at handling high cardinality variables, which is especially valuable when look at pre-pitch predictors that contain a mix of continuous and discrete variables. GUIDE trees as a result, are highly interpretable and therefore can be used easily by a hypothetical batter to help better determine what pitch is being thrown and therefore assist in batter decision making. As a result, batter's are more likely to find success and increase their team's chance of success.

1 Introduction

Major League hitters prepare for opposing starters by reviewing scouting reports that summarize the pitcher's repertoire and tendencies. Modern Statcast pitch-by-pitch data make it possible to model these tendencies empirically, but the resulting model is only useful if a hitter can read and internalize it in the minutes before an at-bat. This constraint argues strongly against black-box models for the scout sheet itself, even when those models predict more accurately. Therefore, I recommend the use of GUIDE to create classification trees which produces a single interpretable tree with provably unbiased variable selection,.

The subject of this report is Matthew Boyd, a left-handed starter for the Chicago Cubs whose 2025 repertoire is dominated by a four-seam fastball, a slider, a changeup, a sinker, and an occasional curveball.

2 Data and Preprocessing

Pitch-by-pitch data were retrieved for free via Baseball Savant a repository for baseball statistics, filtered to Boyd's 2025 regular-season appearances, and reduced to one row per pitch. The response variable is `pitch_type` with five levels (FF, SI, SL, CH, CU).

Several preprocessing steps materially affected the resulting tree:

- **Columns dropped.** The columns `des` was removed because it contained a detailed description of the whole play and provided no value to analysis as well as violating GUIDE's 200 character limit.
- **Base state created.** The raw `on_1b`, `on_2b`, and `on_3b` columns store MLBAM player IDs when occupied and are `NA` otherwise, which is unusable as either a numeric or a categorical predictor. I combined them into a single three-level categorical `base_state` variable (empty / runner not in scoring position / RISP) and excluded the raw columns in the `.dsc` file (see listing 2).
- **Non-predictor variables excluded.** All variables that were not pre-pitch data such as `launch_angle` and `exit_velocity` were assigned an x in the `.dsc` because they weren't valuable in predicting `pitch_type`
- **"Unrealistic" predictors excluded.** Variables that had produced suspicious splits in an exploratory tree, `bat_win_exp`, `if_fielding_alignment`, `of_fielding_alignment`, `batter_days_since_prev_` and `bat_score` were changed to an x in the `.dsc`. These variables are correlated with pitch selection but are not features a pitcher consciously uses; allowing the tree to split on them produced branches that did not reflect the pitcher's actual tendencies.

3 The GUIDE Model

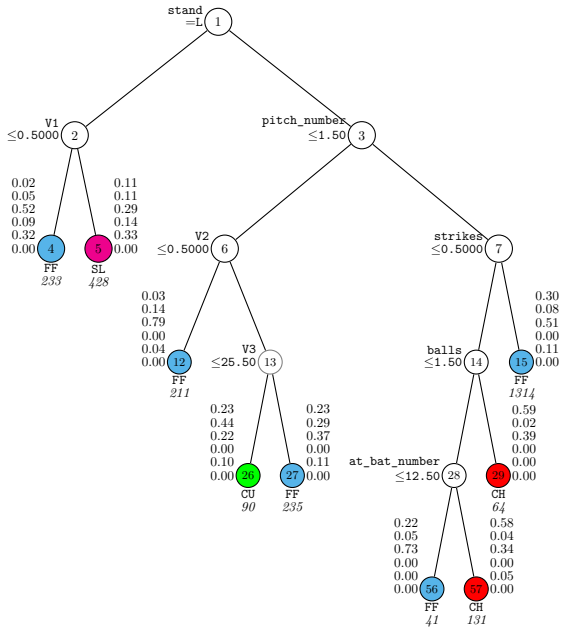
3.1 Algorithm and Configuration

GUIDE grows a classification tree by selecting splitting variables using a χ^2 test of independence between residuals and predictor categories, which removes the variable selection bias inherent in CART/`rpart`. After selecting the variable, the split point is chosen by exhaustive search. Pruning is performed by 10-fold cross-validation, with the tree size selected by the 1-SE rule.

The configuration used (full input file in listing 3): single tree (not ensemble); classification; estimated priors; unit misclassification costs; CV pruning at 1.0 SE; univariate splits at highest priority; maximum 5 levels; minimum node size 30; LaTeX tree output enabled; R prediction function exported.

3.2 The Tree

Figure 1 shows the pruned tree. Each terminal node displays the posterior class probabilities in the order (CH, CU, FF, SI, SL, none) and the sample size in italics.



GUIDE v.46.2 1.000-SE classification tree for predicting `pitch_type` using estimated priors and unit misclassification costs. At each split, an observation goes to the left branch if and only if the condition is satisfied. `V1` = `n_prior_thisgame_player_at_bat`. `V2` = `n_prior_thisgame_player_at_bat`. `V3` = `age_bat_legacy`. Splits at nodes drawn with gray circles are not statistically significant. Predicted class and sample size (in *italics*) printed below each terminal node; class posterior probabilities for `pitch_type` = `CH`, `CU`, `FF`, `SI`, `SL`, and `none`, respectively, beside node. Second best split variable at root node is `pitch_number`.

Figure 1: GUIDE classification tree for Boyd's 2025 pitch selection. GUIDE input in listing 3

Interpreting GUIDE tree

Root split: `stand` (batter handedness). The most important single driver of Boyd's pitch selection.

Versus left-handed batters the next split is on the batter's prior plate appearances in the game (`n_prior_thisgame_player_at_bat`). First time facing a lefty, Boyd is fastball-dominant (∼52% FF, ∼32% SL, with offspeed pitches near zero). Second time or later the slider becomes the go to pitch (∼33%), the fastball drops to ∼29%, and the curveball and sinker each climb past 10%. The interpretation is straightforward: once a left-handed hitter has timed Boyd's fastball, he diversifies aggressively.

Versus right-handed batters the next split is on `pitch_number` $\leq$ `1.50`, separating the first pitch of an at-bat from later pitches. First-pitch, first-time-through against a righty, Boyd is heavily fastball (∼79% FF). On later pitches and later at-bats, the changeup gets established, with the FF/CH combination dominating once a strike has been established. Against younger batters, the curveball is used more often, likely trying to fool them with a ball that has high break.

Figure 2 shows pitch proportions in intermediate nodes for righties. It follows a GUIDE tree path that allows to follow how GUIDE chose the splits

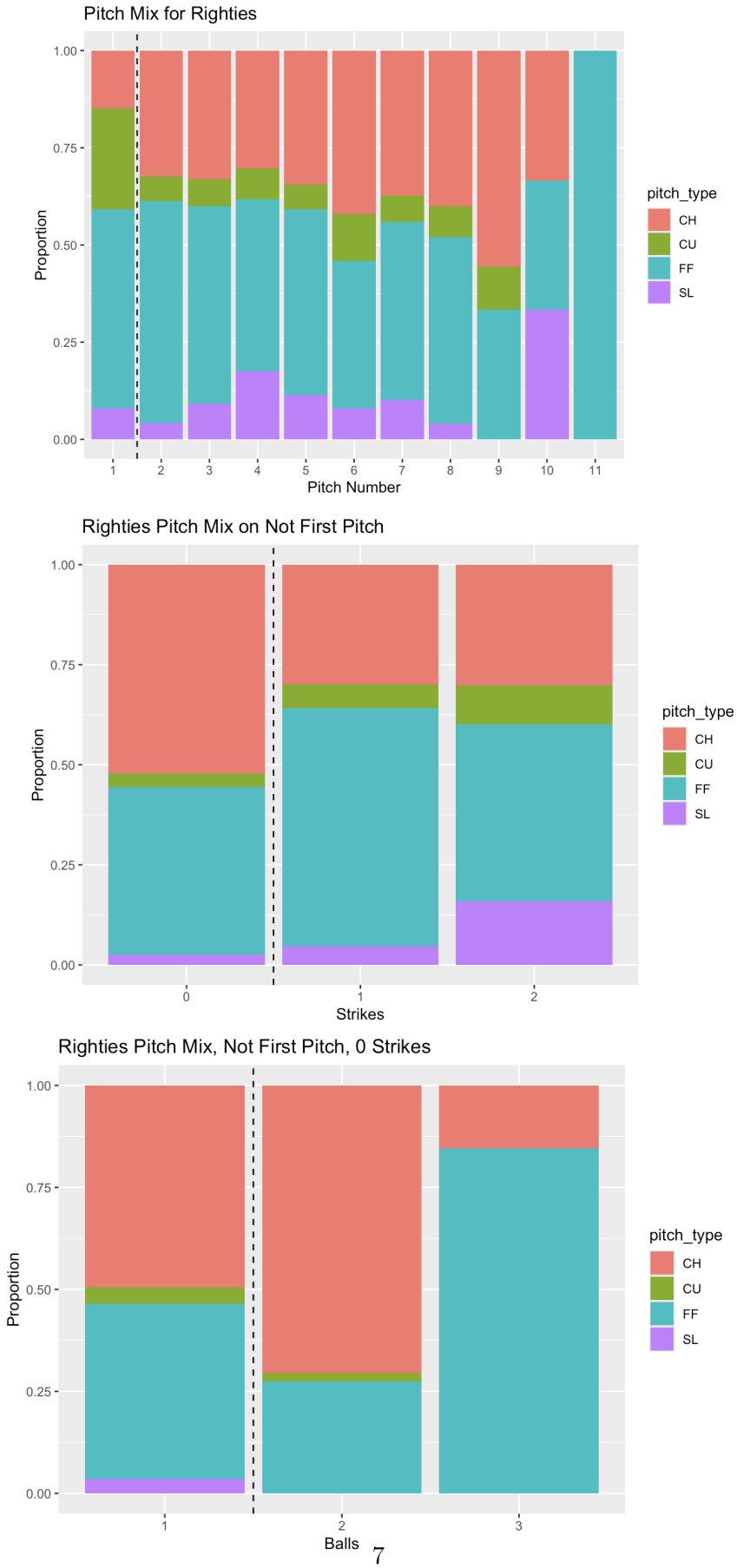


Figure 2: Division of righties pitch types with dashed line indicating GUIDE split

4 Model Comparison

4.1 rpart

Figure 3 is the output of the r code that produces `rpart`. It shares many similarities with GUIDE, with deviation on split and keeping very similar predictors.

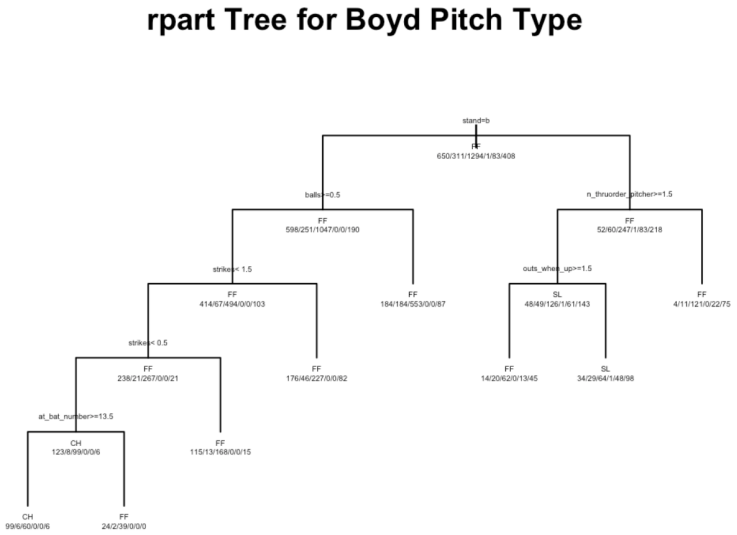


Figure 3: rpart tree. Code to reproduce in listing 5

4.2 ctree

Figure 4 is the output of the r code that produces `ctree`. It shares many similarities with GUIDE, with deviation on split and keeping very similar predictors.

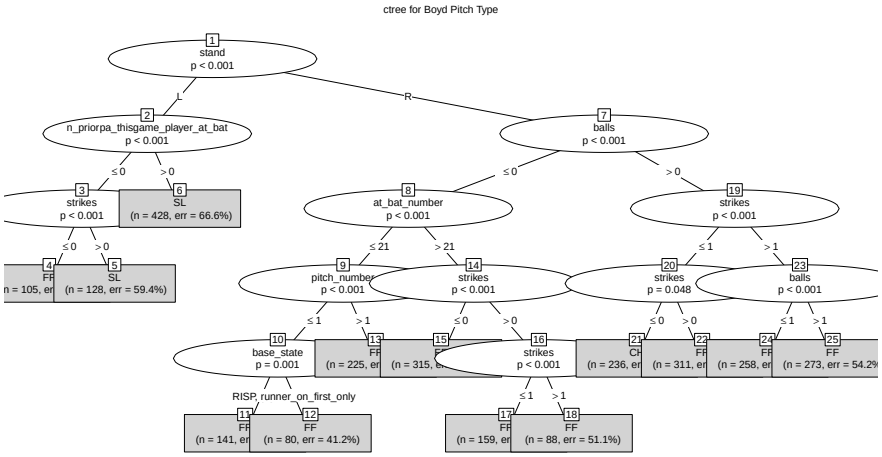


Figure 4: ctree tree. Code to reproduce in listing 6

4.3 ranger

Figure 5 is the ranger random forest simulation of variable importance. High cardinality variables such as `at_bat_number` and `age_bat_legacy` are given much more importance in this model as compared to the more discrete variables such as `balls` and `strikes`.

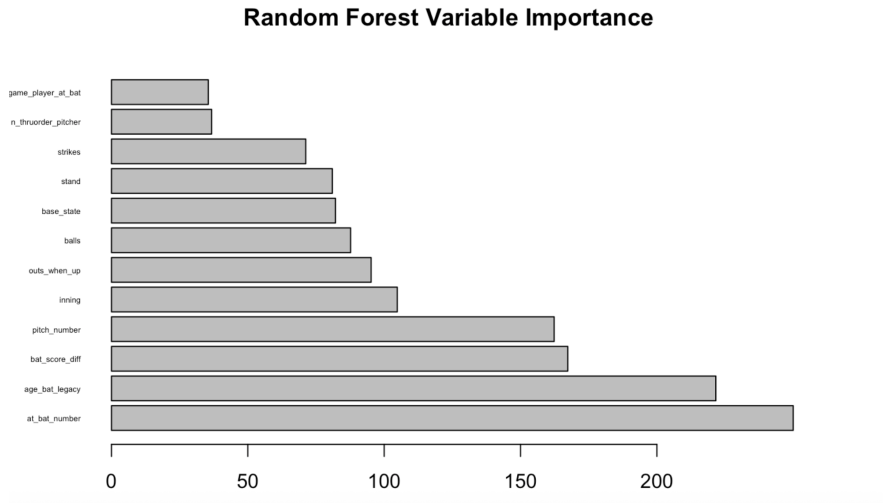
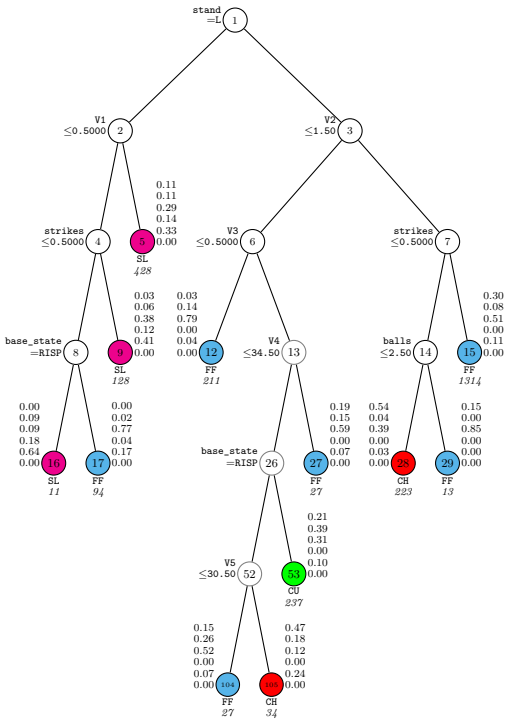


Figure 5: Ranger importance plot. Importance goes from least to most as read from top to bottom. Code to reproduce in listing 7.

4.4 GUIDE LDA

Figure 6 is the GUIDE LDA tree. It performs well but still has a higher misclassification rate than GUIDE simple due do the assumption violations the variables make as a result of their very discrete and non-normal nature of the data.



GUIDE v.46.2 0.250-SE classification tree for predicting `pitch_type` using estimated priors and unit misclassification costs. At each split, an observation goes to the left branch if and only if the condition is satisfied. `V1` = `n_prior_thisgame_player_at_bat`. `V2` = `pitch_number`. `V3` = `n_prior_thisgame_player_at_bat`. `V4` = `age_bat_legacy`. `V5` = `at_bat_number`. Splits at nodes drawn with gray circles are not statistically significant. Predicted class and sample size (in *italics*) printed below each terminal node; class posterior probabilities for `pitch_type` = `CH`, `CU`, `FF`, `SI`, `SL`, and `none`, respectively, beside node. Second best split variable at root node is `pitch_number`.

Figure 6: GUIDE LDA. GUDIDE input in listing 8

4.5 Model comparison by misclassification rate

I compared the GUIDE simple tree against five competing methods using 500 random 80/20 train/test splits, recording the misclassification error on each test set. The methods are: GUIDE with LDA node models, `rpart`, `ctree`, multinomial logistic regression, and `ranger` (random forest).

Section 4.5 shows the distribution of test errors across the 500 replications for each method.

Method	Mean error	Median error	SD
`rpart`	0.530	0.531	0.019
`ctree`	0.530	0.529	0.019
Multinomial logistic	0.52	0.525	0.021
Random forest (`ranger`)	0.568	0.567	0.019

Table 1: Summary of the 500-replication misclassification rate simulation.

Section 4.5 shows the box and whisker plot for all methods tested with GUIDE simple and GUIDE LDA misclassification rates are pulled from `output.txt`. One thing to note is that misclassification rate is high for all models which is to be expected as the models are trying to predict human decision making which has high variation by nature. GUIDE simple performs the best of all the models on average.

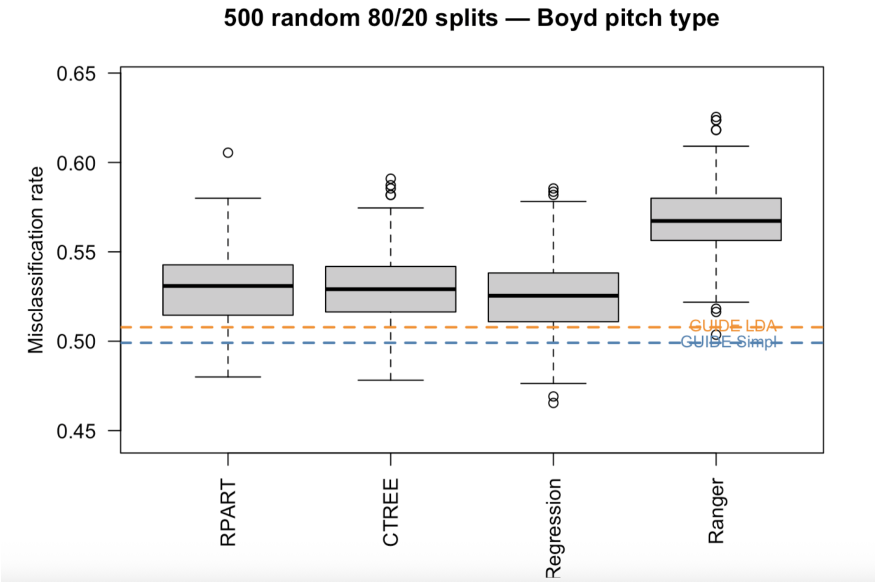


Figure 7: Box-and-whisker plot for all tested methods. Code to reproduce in listing 9.

5 Conclusion

When attempting to predict Matthew Boyd's pitch selections, GUIDE out performed black-box methods by having lower misclassification rates and are easily interpretable. GUIDE and its χ^2 method is more immune to carnality as compared to other variables and therefore excels with `pitch_type` predictors.

A Code for Figures

This appendix collects the R and GUIDE code chunks used to produce each figure in the main body. Each listing is referenced from its corresponding figure caption.

```

boyd = read.csv("boyd_2025.csv")
boyd$events[is.na(boyd$events) | boyd$events == ""] = "none"
boyd$description[is.na(boyd$description) | boyd$description == ""] = "none"
boyd$bb_type[is.na(boyd$bb_type) | boyd$bb_type == ""] = "none"
boyd$hit_location[is.na(boyd$hit_location) | boyd$hit_location == ""] = "none"
for (col in names(boyd)) {
  if (is.character(boyd[[col]]) | is.factor(boyd[[col]])) {
    boyd[[col]] <- as.character(boyd[[col]])
    boyd[[col]][is.na(boyd[[col]]) | boyd[[col]] == ""] <- "none"
  }
}
drop_cols = c("des")
```

Listing 1: Data cleansing

```

boyd$events[is.na(boyd$events) | boyd$events == ""] = "none"
boyd$description[is.na(boyd$description) | boyd$description == ""] = "none"
boyd$bb_type[is.na(boyd$bb_type) | boyd$bb_type == ""] = "none"
boyd$hit_location[is.na(boyd$hit_location) | boyd$hit_location == ""] = "none"
```

Listing 2: Creating the `base_state` categorical variable from raw Statcast on-base ID columns.

```
GUIDE       (do not edit this file unless you know what you are doing)
  46.2      (version of GUIDE that generated this file)
 1          (1=model fitting, 2=importance or DIF scoring, 3=data conversion)
"boyd_type_out_lda.txt"  (name of output file)
 1          (1=one tree, 2=ensemble)
 1          (1=classification, 2=regression, 3=propensity score tree)
 1          (1=simple model, 2=nearest-neighbor, 3=kernel, 4=lda models, 5=qda
    models)
 1          (0=linear 1st, 1=univariate 1st, 2=skip linear, 3=skip linear and
    interaction)
 1          (0=tree with fixed no. of nodes, 1=prune by CV, 2=by test sample,
    3=no pruning)
"boyd_type.dsc"  (name of DSC file)
        10  (number of cross-validations)
 1          (1=mean-based CV tree, 2=median-based CV tree)
  1.000     (SE number for pruning)
 1          (1=estimated priors, 2=equal priors, 3=other priors)
 1          (1=unit misclassification costs, 2=other)
 2          (1=split point from quantiles, 2=use exhaustive search)
 2          (1=default max. number of split levels, 2=specify no. in next line)
 5          (max. number of split levels)
 2          (1=default min. node size, 2=specify min. value in next line)
 30         (min. node size)
 2          (0=no LaTeX code, 1=tree without node numbers, 2=tree with node
    numbers)
"boyd_type_tree.tex" (latex file name)
 1          (1=color terminal nodes, 2=no colors)
 3          (0=highest posterior, 1=sample sizes, 2=sample proportions,
    3=posterior probs, 4=nothing)
```

```
 1          (1=no storage, 2=store fit and split variables, 3=store split
    variables and values)
 2          (1=do not save fitted values and node IDs, 2=save in a file)
"boyd_type_ids" (file name for fitted values and node IDs)
 2          (1=skip R function, 2=R function with train file name, 3=R function
    with file name of new data)
"boyd_type_rfunc" (R code file)
 1          (rank of top variable to split root node)
```

Listing 3: GUIDE input file used to fit the tree in fig. 1. Save as `boyd_type_in.txt` and run with `guide < boyd_type_in.tx`.

```
#pitch mix by first pitch vs not
ggplot(boyd_r, aes(x = factor(pitch_number), fill = pitch_type)) +
  geom_bar(position = "fill") +
  geom_vline(xintercept = 1.5, linetype = "dashed") +
  labs(x = "Pitch Number", y = "Proportion", title = "Pitch Mix for Righties")
  #non first pitch mix by number of strikes
boyd_r_notfirst = boyd_r |>
  filter(boyd_r$pitch_number > 1)
ggplot(boyd_r_notfirst, aes(x = factor(strikes), fill = pitch_type)) +
  geom_bar(position = "fill") +
  geom_vline(xintercept = 1.5, linetype = "dashed") +
  labs(x = "Strikes", y = "Proportion", title = "Righties Pitch Mix on Not First
      Pitch")
  #0 Strikes on non first pitch for righties
boyd_r_notfirst_0k = boyd_r_notfirst |>
  filter(boyd_r_notfirst$strikes == 0)
ggplot(boyd_r_notfirst_0k, aes(x = factor(balls), fill = pitch_type)) +
  geom_bar(position = "fill") +
  geom_vline(xintercept = 1.5, linetype = "dashed") +
  labs(x = "Balls", y = "Proportion", title = "Righties Pitch Mix, Not First
      Pitch, 0 Strikes")
```

Listing 4: Code for fig. 2: tree splits for righties.

```
boyd_rpart = rpart(pitch_type ~ ., data = boyd_preds, method = "class",
                   control = rpart.control(cp = 0.005, minsplit = 30))

plot(boyd_rpart, uniform = TRUE, margin = 0.1)
text(boyd_rpart, use.n = TRUE, all = TRUE, cex = 0.3)
title("rpart Tree for Boyd Pitch Type")
```

Listing 5: Code for fig. 3: rpart tree.

```
boyd_ctree = ctree(pitch_type ~., data = boyd_preds,
           control = ctree_control(minbucket = 30))

pdf("ctree_plot.pdf", width = 14, height = 8)
plot(boyd_ctree, type = "simple",
     main = "ctree for Boyd Pitch Type")
dev.off()
```

Listing 6: Code for fig. 4: ctree tree

```
boyd_ranger = ranger(pitch_type ~ ., data = boyd_preds, num.trees = 500,
                     classification = TRUE, importance = "impurity")
```

```

importance_ranger = sort(boyd_ranger$variable.importance, decreasing = TRUE)
barplot(importance_ranger, horiz = TRUE, las = 1, cex.names = 0.4,
        main = "Random Forest Variable Importance" )
```

Listing 7: Code for fig. 5.

```
GUIDE       (do not edit this file unless you know what you are doing)
  46.2      (version of GUIDE that generated this file)
 1          (1=model fitting, 2=importance or DIF scoring, 3=data conversion)
"lda_out2.txt"  (name of output file)
 1          (1=one tree, 2=ensemble)
 1          (1=classification, 2=regression, 3=propensity score tree)
 4          (1=simple model, 2=nearest-neighbor, 3=kernel, 4=lda models, 5=qda
    models)
 1          (0=tree with fixed no. of nodes, 1=prune by CV, 2=by test sample,
    3=no pruning)
"boyd_type.dsc"  (name of DSC file)
        10  (number of cross-validations)
 1          (1=mean-based CV tree, 2=median-based CV tree)
  1.000     (SE number for pruning)
 1          (1=estimated priors, 2=equal priors, 3=other priors)
 1          (1=unit misclassification costs, 2=other)
 1          (1=default max number of non-categorical splits, 2=specify number in
    next line)
 2          (1=default max. number of split levels, 2=specify no. in next line)
        5   (max. no. split levels)
 2          (1=default min. node size, 2=specify min. value in next line)
       30   (min. node sample size)
 2          (0=no LaTeX code, 1=tree without node numbers, 2=tree with node
    numbers)
"lda_tree2.tex" (latex file name)
 1          (1=color terminal nodes, 2=no colors)
 4          (0=highest posterior, 1=sample sizes, 2=sample proportions,
    3=posterior probs, 4=misclassification cost, 5=nothing)
 2          (1=no storage, 2=store discriminant coefficients)
"lda_coefs.txt" (LDA coefs file name)
 2          (1=do not save fitted values and node IDs, 2=save in a file)
"lda_fit.txt" (file name for fitted values and node IDs)
 2          (1=skip R function, 2=R function with train file name, 3=R function
    with file name of new data)
"lda.r" (R code file)
 1          (1=output R code for plots, 2=skip this)
"lda.plot.r" (R plot file)
"lda_data.txt" (file name for plot data)
```

Listing 8: `input.txt` for GUIDE LDA tree

```
set.seed(2016)
n_reps = 500
train_frac = 0.8

method_names = c("RPART", "CTREE", "Logistic Regression", "Ranger")
errors = matrix(NA, nrow = n_reps, ncol = length(method_names),
                dimnames = list(NULL, method_names))

for(i in seq_len(n_reps)) {
  idx = sample(seq_len(nrow(boyd_preds)), floor(train_frac * nrow(boyd_preds)))
```

```
  train = boyd_preds[idx, ]
  test = boyd_preds[-idx, ]

  train$pitch_type = droplevels(train$pitch_type)
  test$pitch_type = droplevels(test$pitch_type)

  truth = as.character(test$pitch_type)

  fit = rpart(pitch_type ~ ., data = train, method = "class")
  errors[i, "RPART"] = mean(predict(fit, test, type = "class") != test$pitch_type)

  fit = ctree(pitch_type ~ ., data = train)
  errors[i, "CTREE"] = mean(predict(fit, test) != test$pitch_type)

  fit = tryCatch(multinom(pitch_type ~ ., data = train, trace = FALSE),
                 error = function(e) NULL)
  if(!is.null(fit))
    errors[i, "Logistic Regression"] = mean(predict(fit, test) != test$pitch_type)

  fit = ranger(pitch_type ~ ., data = train, num.trees =500,
               classification = TRUE)
  errors[i, "Ranger"] = mean(predict(fit, test)$predictions != test$pitch_type)

  if(i %% 50 == 0) cat("Replication", i, "of", n_reps, "\n")
}
```

Listing 9: Code for section 4.5 and table 1: 500 method simulations.

```
guide_chisq_err = 0.49908992
guide_lda_err   = 0.50782672

boxplot(errors,
        ylab = "Misclassification rate",
        main = paste0("500 random ", round(train_frac*100), "/",
                      round((1-train_frac)*100), " splits     Boyd pitch type"),
        col  = "grey80",
        las  = 2,
        ylim = c(min(errors, guide_chisq_err, guide_lda_err, na.rm = TRUE) - 0.02,
                 max(errors, na.rm = TRUE) + 0.02))

# GUIDE simple-model reference line
abline(h = guide_chisq_err, col = "steelblue", lwd = 2, lty = 2)
text(x = ncol(errors) + 0.45, y = guide_chisq_err,
     labels = "GUIDE Simpl", col = "steelblue", pos = 2, cex = 0.8)

# GUIDE LDA reference line
abline(h = guide_lda_err, col = "darkorange", lwd = 2, lty = 2)
text(x = ncol(errors) + 0.45, y = guide_lda_err,
     labels = "GUIDE LDA", col = "darkorange", pos = 2, cex = 0.8)
```

Listing 10: Code for section 4.5 boxplot.